

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Вычислительная математика»

Студент гр. 4343

Едина А.С.

Преподаватель

Пуеров Г. Ю.

Санкт-Петербург

2026

Цель работы

Изучение составных формул прямоугольников, трапеций и Симпсона для численного вычисления определенных интегралов. Реализация этих методов на языке C++ и анализ их точности на примере интеграла

$$\int_0^1 e^{\cos(x)}$$

Теоретическая часть

Требуется вычислить определенный интеграл

$$I = \int_b^a f(x) dx$$

где $f(x)$ — непрерывная функция на отрезке $[a, b]$. Для многих функций первообразная не выражается через элементарные функции, поэтому применяются численные методы.

Все рассматриваемые методы основаны на разбиении отрезка $[a, b]$ на n равных частей с шагом

$$h = \frac{b - a}{n}$$

и замене интеграла суммой площадей простых геометрических фигур.

Метод прямоугольников

На каждом частичном отрезке функция заменяется постоянным значением в средней точке.

$$I \approx h \sum_{i=0}^{n-1} f\left(x_i + \frac{h}{2}\right)$$

Порядок точности: $O(h^2)$

Формула погрешности:

$$R = \frac{|I_{2n} - I_n|}{3}$$

Метод трапеций

На каждом частичном отрезке функция заменяется линейной (хордой), площадь под кривой аппроксимируется площадью трапеции.

$$I \approx \frac{h}{2} \sum_{i=0}^{n-1} (f(x_i) + f(x_{i+1}))$$

Порядок точности: $O(h^2)$

Формула погрешности:

$$R = \frac{|I_{2n} - I_n|}{3}$$

Метод Симпсона

На каждом сдвоенном отрезке функция заменяется параболой (многочленом второй степени), проходящей через три точки.

$$I \approx \frac{h}{3} \sum_{i=0}^{m-1} (f(x_{2i}) + 4f(x_{2i+1}) + f(x_{2i+2}))$$

где n должно быть четным.

Порядок точности: $O(h^4)$ — самый высокий среди рассмотренных методов.

Формула погрешности:

$$R = \frac{|I_{2n} - I_n|}{15}$$

Оценка погрешности по правилу Рунге

Правило Рунге позволяет оценить погрешность численного метода без знания точного значения интеграла. Для этого сравниваются результаты при разбиениях с шагом h и $\frac{h}{2}$ (т.е. при n и $2n$).

Подбор числа разбиений

Число разбиений n влияет на точность и скорость вычислений:

- Слишком малое n – большая погрешность
- Слишком большое n – медленная работа

Поэтому n подбирается автоматически: начиная с минимального значения, оно удваивается до тех пор, пока оценка погрешности не станет меньше заданной точности ε .

Исследуемая функция

Вычисляется интеграл:

$$I = \int_0^1 e^{\cos(x)}$$

Точное значение (с высокой точностью): $I \approx 2.341574$

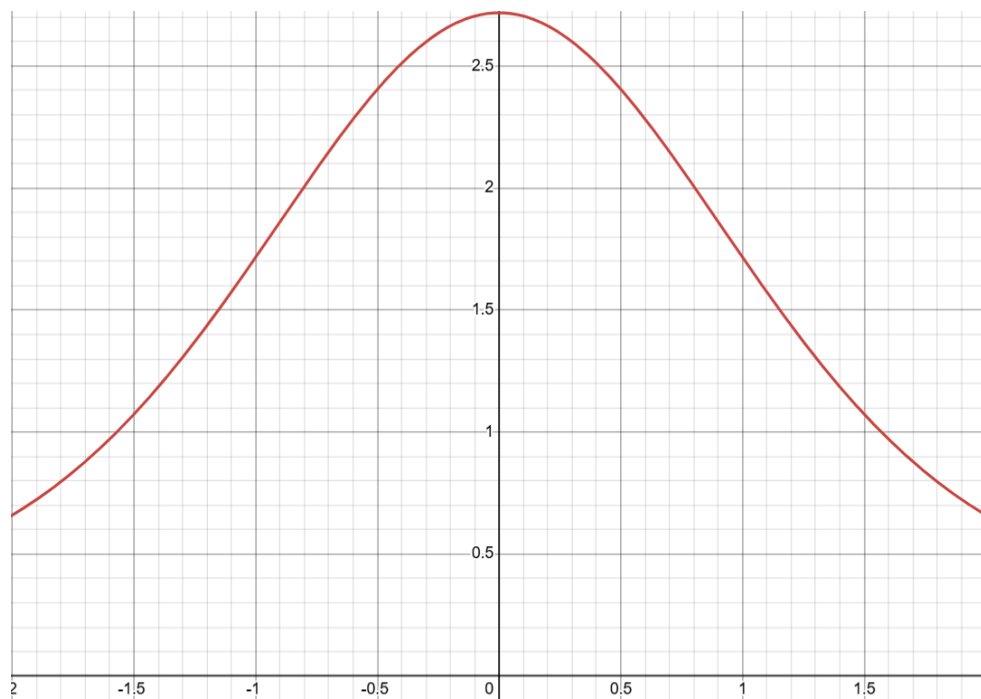


Рисунок 1 - График функции $e^{\cos(x)}$

Тестирование программы

Таблица 1 - Результаты тестирования

Точность	Метод	Число отрезков	Значение интеграла	Оценка погрешности
0.1	Прямоугольников	2	2.35683	0.0038277
	Трапеций	2	2.31124	0.00759857
	Симпсона	2	2.34252	5.89899e-05
0.001	Прямоугольников	4	2.34535	0.000944479
	Трапеций	8	2.33969	0.000470477
	Симпсона	2	2.34252	5.89899e-05
0.0001	Прямоугольников	16	2.34181	5.87891e-05
	Трапеций	32	2.34146	2.93877e-05
	Симпсона	2	2.34252	5.89899e-05
0.00001	Прямоугольников	64	2.34159	3.67338e-06
	Трапеций	64	2.34155	7.3467e-06
	Симпсона	4	2.34164	3.7891e-06
0.000001	Прямоугольников	128	2.34158	9.18333e-07
	Трапеций	256	2.34157	4.59165e-07
	Симпсона	8	2.34158	2.34933e-07

Вывод

Изучены составные формулы численного интегрирования: методы прямоугольников, трапеций и Симпсона. Реализованы алгоритмы этих методов на языке C++. Проведен сравнительный анализ методов на примере интеграла. Установлено, что метод Симпсона обладает наивысшей точностью и требует наименьшего числа разбиений для достижения заданной погрешности.

Приложения А

Исходный код main.cpp

```
#include <iostream>
#include <cmath>

using namespace std;

double f(double x) {
    return exp(cos(x));
}

double rect(double a, double b, int n) {
    double h = (b - a) / n;
    double S = 0.0;
    for (int i = 0; i < n; i++) {
        double x_mid = a + (i + 0.5) * h;
        S += f(x_mid);
    }
    return S * h;
}

double trap(double a, double b, int n) {
    double h = (b - a) / n;
    double S = (f(a) + f(b)) / 2.0;
    for (int i = 1; i < n; i++) {
        S += f(a + i * h);
    }
    return S * h;
}

double simpson(double a, double b, int n) {
    if (n % 2 != 0) n++;
    double h = (b - a) / n;
    double sum = f(a) + f(b);
    for (int i = 1; i < n; i++) {
        double x = a + i * h;
        sum += (i % 2 == 0) ? 2 * f(x) : 4 * f(x);
    }
    return (h / 3.0) * sum;
}

double runge_error_rect(double a, double b, int n) {
    double I_n = rect(a, b, n);
    double I_2n = rect(a, b, 2 * n);
    return fabs(I_2n - I_n) / 3.0;
}

double runge_error_trap(double a, double b, int n) {
    double I_n = trap(a, b, n);
    double I_2n = trap(a, b, 2 * n);
    return fabs(I_2n - I_n) / 3.0;
}

double runge_error_simpson(double a, double b, int n) {
    double I_n = simpson(a, b, n);
    double I_2n = simpson(a, b, 2 * n);
}
```



```

        return fabs(I_2n - I_n) / 15.0;
    }

    int find_n_rect(double a, double b, double eps) {
        int n = 2;
        while (runge_error_rect(a, b, n) > eps) {
            n *= 2;
        }
        return n;
    }

    int find_n_trap(double a, double b, double eps) {
        int n = 2;
        while (runge_error_trap(a, b, n) > eps) {
            n *= 2;
        }
        return n;
    }

    int find_n_simpson(double a, double b, double eps) {
        int n = 2;
        while (runge_error_simpson(a, b, n) > eps) {
            n *= 2;
        }
        return n;
    }

    int main() {
        double a = 0.0, b = 1.0;
        double eps;

        cout << "вычисление интеграла от exp(cos(x)) на [0, 1]" << endl;
        cout << "введите точность: ";
        cin >> eps;

        cout << endl;

        int n_rect = find_n_rect(a, b, eps);
        int n_trap = find_n_trap(a, b, eps);
        int n_simp = find_n_simpson(a, b, eps);

        double rec = rect(a, b, n_rect);
        double trapp = trap(a, b, n_trap);
        double simp = simpson(a, b, n_simp);

        cout << "метод прямоугольников:" << endl;
        cout << "| требуется отрезков: " << n_rect << endl;
        cout << "| значение: " << rec << endl;
        cout << "| оценка погрешности: " << runge_error_rect(a, b,
n_rect) << endl;
        cout << endl;

        cout << "метод трапеций:" << endl;
        cout << "| требуется отрезков: " << n_trap << endl;
        cout << "| значение: " << trapp << endl;
        cout << "| оценка погрешности: " << runge_error_trap(a, b,
n_trap) << endl;
        cout << endl;
    }

```

```
        cout << "метод Симпсона:" << endl;
        cout << "| требуется отрезков: " << n_simp << endl;
        cout << "| значение: " << simp << endl;
        cout << "| оценка погрешности: " << runge_error_simpson(a, b,
n_simp) << endl;

        return 0;
    }
```